# LeetCode HOT 100

## PART 3

## Sliding Window

## 滑动窗口

# 8. 无重复字符的最长子串 (No.3) MEDIUM

给你一个整数数组 `nums` ，判断是否存在三元组 `[nums[i], nums[j], nums[k]]` 满足 `i != j`、`i != k` 且 `j != k` ，同时还满足 `nums[i] + nums[j] + nums[k] == 0` 。请你返回所有和为 `0` 且不重复的三元组。

**注意：** 答案中不可以包含重复的三元组。

**示例 1：**

> **输入：** nums = [-1,0,1,2,-1,-4]
> **输出：** [[-1,-1,2],[-1,0,1]]
> **解释：**
> nums[0] + nums[1] + nums[2] = (-1) + 0 + 1 = 0 。
> nums[1] + nums[2] + nums[4] = 0 + 1 + (-1) = 0 。
> nums[0] + nums[3] + nums[4] = (-1) + 2 + (-1) = 0 。
> 不同的三元组是 [-1,0,1] 和 [-1,-1,2] 。
> 注意，输出的顺序和三元组的顺序并不重要。

**示例 2：**

> **输入：** nums = [0,1,1]
> **输出：** []
> **解释：** 唯一可能的三元组和不为 0 。

**示例 3：**

> **输入：** nums = [0,0,0]
> **输出：** [[0,0,0]]
> **解释：** 唯一可能的三元组和为 0 。

**提示：**

- `3 <= nums.length <= 3000`
- `-10^5 <= nums[i] <= 10^5`

# 8. 无重复字符的最长子串 (No.3) MEDIUM

**思路**

用一个哈希表记录字符信息，key是字符，value是该字符在s中的下标。当右边界right遇到重复字符，就把左边界left给收缩，缩到重复字符上一次出现地方的右边。

需要小心，如果上一次left已经因为重复字符收缩了，马上right又遇到重复字符，但这个重复字符在此时left的左侧，那么要让left保持不变。因为left只负责收缩窗口，不能扩张窗口，否则又会有重复字符。

```
class Solution(object):
    def lengthOfLongestSubstring(self, s):
        """
        :type s: str
        :rtype: int
        """
        num_map = {}  # key是窗口内字符，value是字符在s中的下标
        res, left, right = 0, 0, 0
        for right in range(len(s)):
            if s[right] in num_map:  # 重复字符
                # 易错：窗口左端移动到重复字符的下一个位置或保持不变
                left = max(num_map[s[right]] + 1, left)
            num_map[s[right]] = right
            res = max(res, right - left + 1)
        return res
```

**TIPS：滑动窗口代码中，外循环扩展右边界，内循环收缩左边界**

时间复杂度：$\mathcal{O}(n)$

空间复杂度：$\mathcal{O}(|\Sigma|)$

$|\Sigma|$表示字符集大小

# 9. 找到字符串中所有字母异位词 (No.438) MEDIUM

给定两个字符串 `s` 和 `p`，找到 `s` 中所有 `p` 的 **异位词** 的子串，返回这些子串的起始索引。不考虑答案输出的顺序。

**示例 1:**

```
输入： s = "cbaebabacd", p = "abc"
输出： [0,6]
解释：
起始索引等于 0 的子串是 "cba", 它是 "abc" 的异位词。
起始索引等于 6 的子串是 "bac", 它是 "abc" 的异位词。
```

**示例 2:**

```
输入： s = "abab", p = "ab"
输出： [0,1,2]
解释：
起始索引等于 0 的子串是 "ab", 它是 "ab" 的异位词。
起始索引等于 1 的子串是 "ba", 它是 "ab" 的异位词。
起始索引等于 2 的子串是 "ab", 它是 "ab" 的异位词。
```

**提示：**

- $1 <= s.length, p.length <= 3 * 10^4$
- `s` 和 `p` 仅包含小写字母

# 9. 找到字符串中所有字母异位词 (No.438) MEDIUM

**Solution 1 – 定长滑窗**

## 思路

维护大小为len(p)的滑动窗口与两个字符计数器，分别负责对p和window进行字符计数，二者相等时出现异位词。注意：(1)窗口大小不足len(p)时直接continue；(2)本代码用python3提交(不是python，老版本Counter对于计数为0的key不会自动删除)，如果用python提交或者不用Counter用哈希表实现，要手动del计数归零的key。

```python
class Solution(object):
    def findAnagrams(self, s, p):
        """
        :type s: str
        :type p: str
        :rtype: List[int]
        """
        p_map, window = Counter(p), Counter()  # p_map和window都是字符计数器
        res, left = [], 0
        # 窗口大小始终为len(p)
        for right in range(len(s)):
            window[s[right]] += 1
            if right < len(p) - 1:
                continue  # 窗口大小不足len(p)
            if window == p_map:  # 窗口内字符串是p的异位词
                res.append(left)
            window[s[left]] -= 1  # 左边界收缩
            left += 1
        return res
```

**时间复杂度**：$\mathcal{O}(|\Sigma|n+m)$

**空间复杂度**：$\mathcal{O}(|\Sigma|)$

$n$表示s长度，$m$表示p长度

$\Sigma$表示字符集

# 9. 找到字符串中所有字母异位词 (No.438) MEDIUM

**Solution 2 – 不定长滑窗**

## 思路

维护不定长滑动窗口与一个字符计数器，负责对p进行字符计数：当right扩展时扣减计数，当因为某字符过多计数为负时收缩left增加计数。窗口大小恰好为len(p)时匹配成功。

```python
class Solution(object):
    def findAnagrams(self, s, p):
        """
        :type s: str
        :type p: str
        :rtype: List[int]
        """
        p_map = Counter(p)
        res, left = [], 0
        for right in range(len(s)):
            p_map[s[right]] -= 1  # 右边界扩展
            while p_map[s[right]] < 0:  # s[right]字符太多了
                p_map[s[left]] += 1
                left += 1  #  左边界收缩
            # 长度为len(p)时匹配成功
            if right - left + 1 == len(p):
                res.append(left)
        return res
```

### 代码中较难理解的两个地方（@灵茶山艾府的证明）

枚举子串 $t$ 的右端点，如果发现 $t$ 其中一种字母的出现次数**大于** $p$ 的这种字母的出现次数，则右移 $t$ 的左端点（缩小窗口）。如果发现 $t$ 的长度等于 $p$ 的长度，则说明 $t$ 的每种字母的出现次数，等于 $p$ 的每种字母的出现次数，即 $t$ 是 $p$ 的异位词。

**证明**：内层循环结束后，$t$ 的每种字母的出现次数，都小于等于 $p$ 的每种字母的出现次数。如果 $t$ 的其中一种字母的出现次数比 $p$ 的小，那么 $t$ 的长度必然小于 $p$ 的长度。所以只要 $t$ 的长度等于 $p$ 的长度，就说明 $t$ 的每种字母的出现次数，和 $p$ 的每种字母的出现次数都相同，$t$ 是 $p$ 的异位词，把 $t$ 左端点下标加入答案。

**问**：为什么内层循环只判断了字母 $c$ 的出现次数，而不是每种字母的出现次数？

**答**：如果字母 $c$ 进入窗口后，窗口不合法（某个 $cnt[x] < 0$），那么罪魁祸首是谁？由于在之前的循环中，我们已经把窗口变成合法的了，所以只能是刚进入窗口的字母 $c$ 导致窗口不合法，其余字母都满足 $cnt[x] \geq 0$，所以只需判断字母 $c$ 的出现次数。

**时间复杂度**：$\mathcal{O}(n+m)$；**空间复杂度**：$\mathcal{O}(|\Sigma|)$